

Introduction to Intelligent Systems

Lecture 13: Generative AI

Andrés Arévalo

Universidad Nacional de Colombia

May 6th, 2026

Agenda

- ① The Generative AI Landscape
- ② How LLMs Work
- ③ Beyond Text: Images, Audio & Video
- ④ Prompt Engineering

References

- Huyen, C. (2025). *AI Engineering*. O'Reilly. Ch. 1–4.
- Grootendorst, M. & Alammar, J. (2025). *An Illustrated Guide to AI Agents*. O'Reilly. Ch. 1–2.

The Generative AI Landscape

What was generated by AI?



Painted a picture

A Wrote a story opening

“La ciudad antigua de Cartagena dormía bajo un cielo violeta, sus calles de adoquín aún tibias del sol de la tarde, mientras Sofía abría por última vez el portón de hierro de la casa de su abuela...”

</> Wrote code

```
def fibonacci(n):  
    if n <= 1: return n  
    return fib(n-1) + fib(n-2)
```



Generated a portrait

Generative vs. Discriminative AI

→ Discriminative AI (what we studied)

Question: which class does this input belong to?

Input $\rightarrow P(y | x) \rightarrow$ **Label**

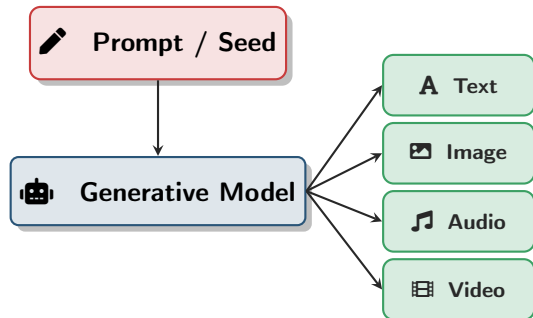
Examples: spam filter, image classifier, decision trees, neural networks.

🔧 Generative AI (today)

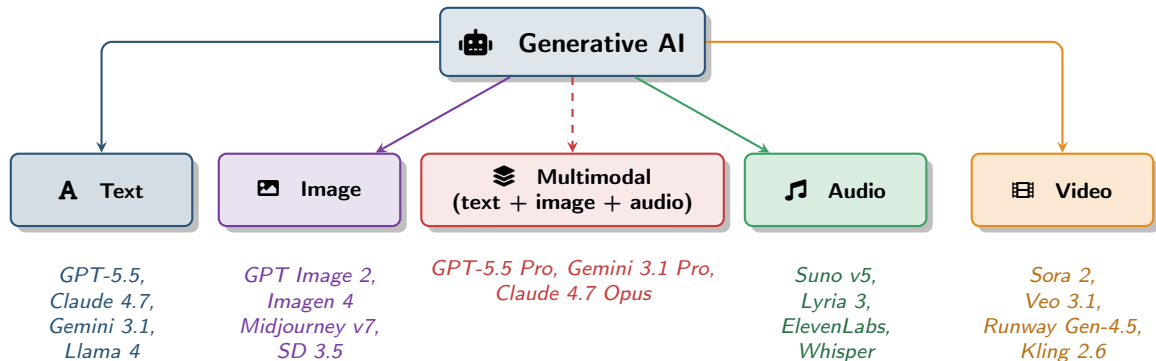
Question: what plausible new content fits this context?

Prompt $\rightarrow P(x) \rightarrow$ **New content**

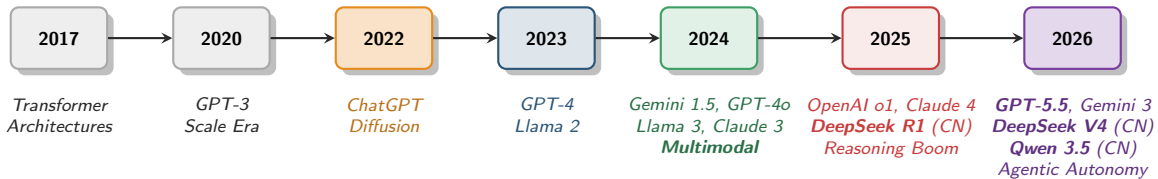
Examples: text, images, code, music, video.



The Generative AI Landscape

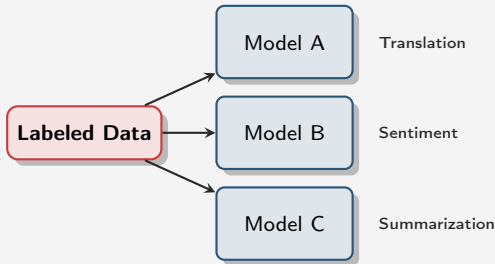


A Decade of Progress: The Global AI Race

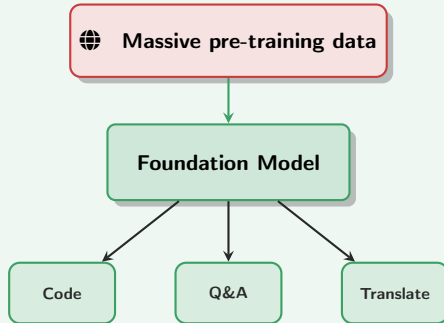


The Foundation Model Paradigm Shift

Before: One Model Per Task



Now: Foundation Model



One model pre-trained once on massive data can be adapted to *thousands* of tasks. The bottleneck shifts from *model design* to **data and compute**.

Llama 4 Behemoth model required 32K GPUs using FP8 and a training data mixture of +30T tokens

Open vs. Closed Models

Closed / Proprietary

- ✓ State-of-the-art performance
- ✓ Maintained & updated by provider
- ✗ Weights not accessible
- ✗ Pay-per-token cost
- ✗ Data leaves your environment

Examples:

GPT-5.5 • Claude Opus 4.7 • Gemini Pro 3.1x

Open / Open-Weight

- ✓ Download & run locally
- ✓ Full control, full privacy
- ✓ Free to use and modify
- ✗ Requires own compute
- ✗ Often behind proprietary SOTA

Examples:

Llama 4 • GLM-5 • Kimi K2.6 • DeepSeek-V4

How LLMs Work

The Secret Ingredient: Attention

The Cocktail Party Analogy

At a noisy party you can still follow one conversation because your brain **selectively focuses** on relevant sounds.

Attention gives LLMs the same superpower: every token can “look at” every other token and decide *how much it matters*.

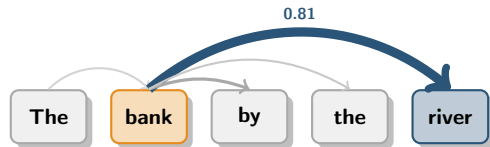
Ambiguity example:

“I went to the **bank**.”

River bank or financial bank?

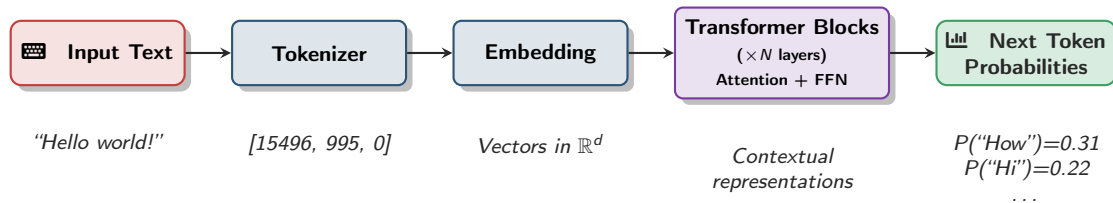
Attention resolves it by weighting nearby words.

Attention from “bank” in “The bank by the river”:



Thicker arrow = stronger attention

Transformers: The Big Picture



LLMs are **next-token predictors**. The magic emerges from doing this *billions* of times on all of human text.

Tokens: The Atomic Unit of LLMs

✗ Myth: LLMs read words

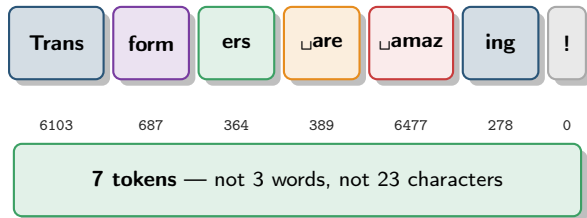
LLMs do **not** split text by spaces.

They use a learned **tokenizer** (BPE) that creates sub-word *tokens* optimized for the training data.

i Why it matters

- API pricing is **per token**, not per word
- Rare words and non-English text cost more tokens
- Prompt + history + output all count against the limit
- ≈ 1 token per 0.75 of an English word

Tokenizing “Transformers are amazing!”



Context Windows: The Model's Working Memory

What is a context window?

The maximum number of tokens the LLM can “see” at once — its **working memory**.

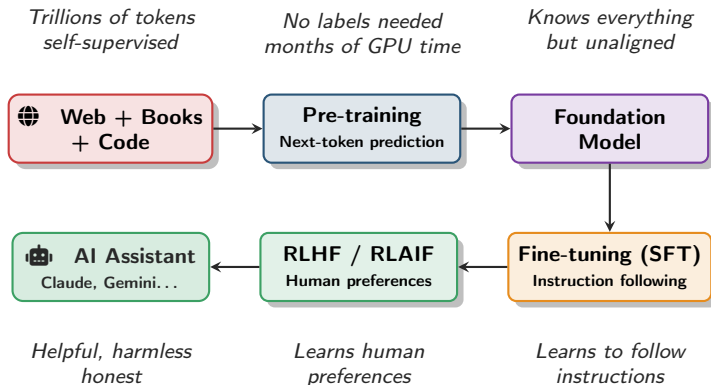
- Everything outside is **forgotten**
- Prompt + history + output all count
- Larger context = more expensive inference

Context window sizes evolution

Model	Tokens
GPT-3 (2020)	2K
GPT-4 (2023)	128K
Claude 3.5 (2024)	200K
Gemini 2.0 Flash (2025)	1M

Tip: keep prompts concise. Exceeding the window causes truncation or sharp quality degradation.

From Raw Data to AI Assistant



Why Bigger Is (Usually) Better

📈 Scaling Laws (Kaplan et al., 2020)

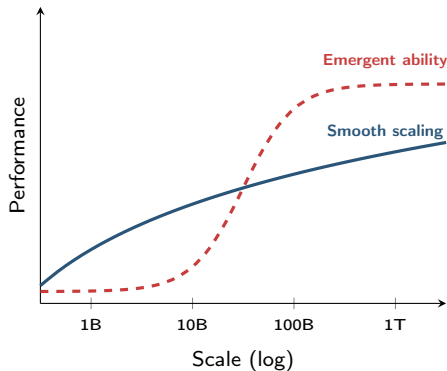
Performance improves *predictably* with three levers:

- **Parameters:** model size
- **Data:** tokens seen during training
- **Compute:** GPU hours (FLOPs)

Follows a **power law** on a log-log scale.
Double the compute → fixed, predictable gain.

⚡ Emergent Abilities

Some skills **appear suddenly** past a scale threshold — not gradual improvement but a phase transition.
Examples: multi-step arithmetic, code generation, chain-of-thought reasoning.



How the Model “Chooses” the Next Token

🌡 Temperature

Scales the probability distribution before sampling.

- **Low (0.1–0.4):** deterministic. Use for code, facts.
- **Medium (0.4–0.9):** balanced conversation.
- **High (≥ 1.0):** creative, unpredictable.

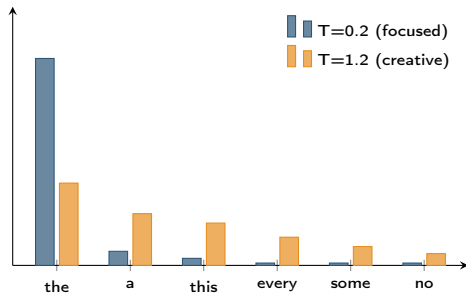
Analogy: a cold chef always makes the same dish; a hot chef improvises.

🔴 Top-p (Nucleus Sampling)

Consider only the smallest set of tokens whose cumulative probability reaches p .

Top-p=0.9: ignore the long tail of unlikely tokens.

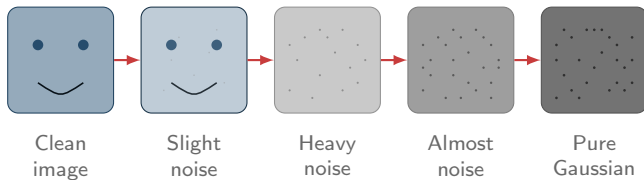
Same prompt, different temperatures:



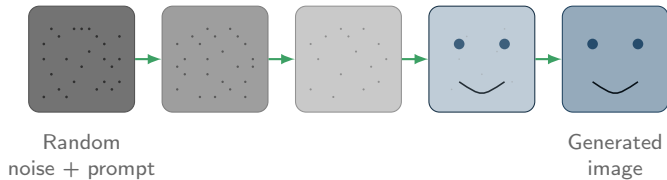
Beyond Text: Images, Audio & Video

How Diffusion Models Generate Images

Forward process (fixed): add Gaussian noise



Reverse process (learned): denoise step by step



The Latent Space: Images as Coordinates



What is a latent space?

A high-dimensional space where each point represents a **compressed version** of an image.

- Similar images are **close** in this space
- Smooth interpolation between points = smooth transition between images
- Text prompts are mapped to the **same** space

💡 Why “latent”?

The meaningful structure is *hidden* (latent) in the high-dimensional representation, not visible directly.

From pixels to meaning

High-dim image
~1M pixels



encoder
 $f(x)$

Latent code
~512 numbers

$z = (z_1, z_2, \dots, z_d)$
0.82 -0.31 1.24 0.07
-1.05 0.46 -0.18 0.93
0.21 0.74 -0.62 -0.40
0.55 -0.88 0.13 0.69
*compact image
summary*

↓ place each code
as a point

latent space

similar images land in similar regions



dogs



cats

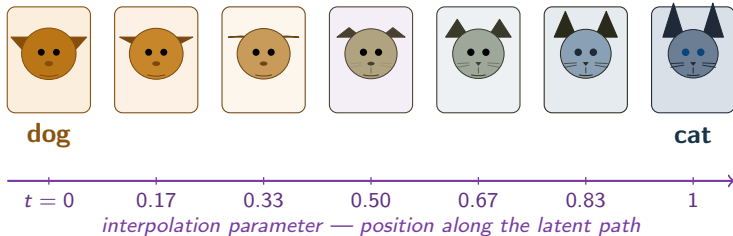


cars

our image's code lives here

Walking the Latent Path: Dog $\rightarrow$ Cat

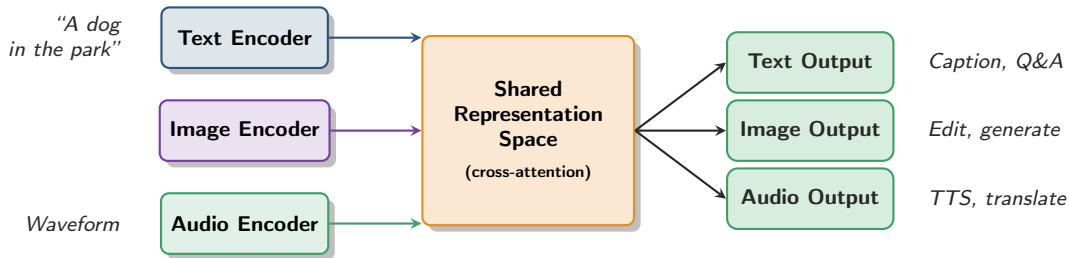
Each tile decodes a point along the interpolation between two embeddings



$$z(t) = (1 - t) \cdot z_{\text{dog}} + t \cdot z_{\text{cat}}$$

Every point in between is a valid embedding the decoder can turn into an image.

Multimodal Models: Many Senses, One Brain



Examples: GPT-5.5 (text+image+audio), Gemini 3.1 Pro (text+image+video+audio), Claude Opus 4.7 (text+image).

Beyond Images: Audio & Video Generation

Audio Generation

Text-to-Speech (TTS):

Convert text to natural voice. Clones voices in seconds.

ElevenLabs, Google TTS, Whisper

Music Generation:

"Generate a lo-fi hip hop track, 120 BPM, melancholic."

Suno, Udio, MusicGen (open source)

Speech-to-Text:

Transcription with near-human accuracy.

Whisper (open source, free)

Video Generation

Text-to-Video:

A prompt becomes a short video clip. Still expensive, improving fast.

Sora (OpenAI), Veo 3 (Google)

Video-to-Video:

Transform an existing video: style transfer, motion retargeting.

Runway, Kling

Status (2025):

Coherent short clips (<60s). Long-form consistency is an open problem.

Deepfakes & synthetic media are a real societal challenge. Detection research is a growing field.

Practitioner's Map of Generative AI Tools (v2026.05)

Modality	Closed (API)	Open / Free
Text	GPT-5.5, Claude 4.7 Opus, Gemini 3.1 Pro	Llama 4 Scout, DeepSeek V4, Qwen 3.6 Max, Mistral 3
Image	Midjourney v7, Imagen 4, DALL-E 4 (GPT Image 2)	FLUX.2, Stable Diffusion 3.5, HiDream-I2
Embeddings	OpenAI text-emb-4, Gemini-E 2.5	BGE-M4, st-v4 (local), Cohere Compass
Audio	Suno v5, ElevenLabs v3, Lyria 3 (Google)	Whisper v4, MusicGen 2, Fish Speech
Video	Sora 2, Veo 3.1, Runway Gen-4 (Agentic)	Kling 2.6, CogVideoX 2, Open-Sora 2.5

Prompt Engineering

Same Model. Completely Different Results.

✗ Naive prompt

Prompt: "Explain neural networks."

Output: A neural network is a computational model inspired by the human brain. It consists of layers of interconnected nodes (neurons)...[continues for 3 pages, technical, no examples]

✓ Engineered prompt

Prompt: "You are a patient teacher. Explain neural networks to a 15-year-old using only a cooking analogy. Be concise: 3 sentences max."

Output: Think of a neural network like a recipe tester. You give it ingredients (data), it tries a dish (prediction), tastes it, and adjusts the recipe (learns) until it's perfect.

💡 **Prompt Engineering** is the skill of designing inputs to get the most useful, accurate, and reliable outputs from an LLM. Same model, totally different value.

Anatomy of a Well-Structured Prompt



System Prompt — Persona, rules, constraints, tone

"You are an expert data scientist. . ."



Context — Background information, retrieved documents, examples

"Here is the dataset description: . . ."



Instruction — The actual task to perform

"Identify the three main data quality issues."



Output Format — JSON, bullet list, length, language

"Return as a numbered list, one sentence each."

Not every prompt needs all four layers. Start with **Instruction**, add the others as you need more control. Use **XML tags** (<context>, <task>) as delimiters for long prompts.

Zero-shot vs. Few-shot Prompting

Zero-shot — No examples

Classify the sentiment:

“The delivery was late and the package arrived damaged.”

Output: Negative
(sometimes unreliable)

Few-shot — Show examples first

Classify as Positive or Negative:

“Great product, love it!” → Positive

“Broken on arrival, waste.” → Negative

“Cold food, rude service.” → Negative

Now classify:

“Fast delivery, works perfectly!”

Output: Positive
(consistent & reliable)

 **Rule of thumb:** 3–5 diverse, well-labeled examples dramatically improve output consistency, especially for classification and formatting tasks. Examples teach the model the *pattern*, not just the task.

Chain-of-Thought (CoT): “Let’s Think Step by Step”

✗ Direct answer (no CoT)

Q: Maria has 15 coins. She spends 3 on a bus, gives 4 to her sister, then her sister returns 2. How many does she have?

A: 10. (may be wrong)

⚡ Direct answers *skip* reasoning.
For multi-step problems this often fails.

✓ With CoT trigger

Q: (same question)...
Think step by step.

A:

Step 1: Start = 15

Step 2: Bus fare: $15 - 3 = 12$

Step 3: Give sister: $12 - 4 = 8$

Step 4: Sister returns: $8 + 2 = 10$

Answer: 10 coins. (reliable)

Just adding “**Think step by step**” or “**Let’s reason carefully**” dramatically improves accuracy on math, logic, and multi-step tasks.

Tree of Thought (ToT): Explore Before Committing

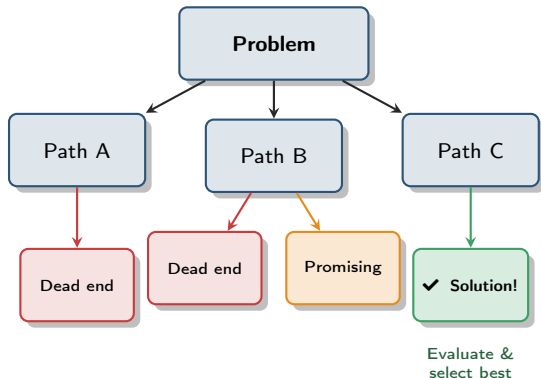


The idea

Instead of one reasoning chain, explore **multiple paths** simultaneously and evaluate each.

- 1 Generate k candidate next steps
- 2 Score each step (self-evaluate)
- 3 Keep the most promising
- 4 Repeat until solution

Best for: puzzles, planning, open-ended problems.



ToT in Practice: A Planning Problem

The Prompt

Schedule tasks A, B, C where C must run after A or B. Each task takes 2-3 hours.

“Generate 3 different orderings.
For each, check feasibility and estimate total time.
Then pick the best and justify.”

Model Output

Path 1: A→B→C ✓ valid, 8h

Path 2: B→A→C ✓ valid, 7h

Path 3: C→A→B ✗ invalid (C first)

Best: Path 2 -- 7h, feasible.

 The key instruction pattern: **generate** k candidates, **evaluate** each, **select** the best. This forces tree-search reasoning in a single prompt.

Self-Consistency: Sample Many, Trust the Majority

🧠 The Core Idea

A single reasoning chain can stumble on an error.

Fix: Send the *same prompt* n times (temperature > 0 so each chain differs independently), then **take the majority vote** on the final answer.

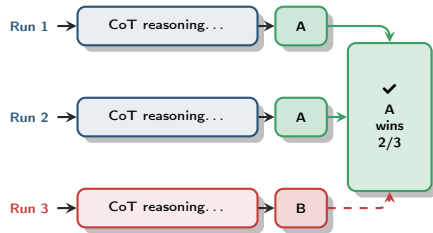
💡 Errors are *diverse*; correct answers *converge*.

✓ Use when

Math, logic, factual Q&A — tasks with a unique correct answer. 3–5 samples are usually enough.

✗ Skip when

Open-ended tasks (writing, brainstorming) — no single “right” answer exists to vote on.



⚡ No extra training. No special model.
Just call the API n times and vote.

Self-Consistency in Action: A Worked Example

“A bookstore sells novels at \$12 and magazines at \$5. Ana buys 3 novels and 4 magazines and pays with \$70. What is her change?” (Correct answer: \$14)

✓ Chain 1

$3 \times \$12 = \36
 $4 \times \$5 = \20
Spent: \$56
 $\$70 - \56
 $= \$14$ ✓

✓ Chain 2

Novels: \$36
Mags: $4 \times \$5 = \20
 $\$36 + \$20 = \$56$
 $\$70 - \56
 $= \$14$ ✓

✗ Chain 3

$3 \times \$12 = \36
 $4 \times \$5 = \18 slip
Spent: \$54
 $\$70 - \54
 $= \$16$ ✗

✓ **Majority vote** $\Rightarrow$ **\$14** (chains 1 & 2 agree) \$16 discarded as the outlier.
No external judge needed — the model's own consistency reveals the correct path.

Role & Persona Prompting

Without persona

Prompt: “What should I do if I have chest pain?”

Output: “Chest pain can have many causes. You should consider seeing a doctor. Common causes include...” [generic, cautious, shallow]

With persona

Prompt: “You are an experienced ER physician explaining triage to a first-year medical student. What are the red-flag symptoms of chest pain that require immediate attention?”

Output: [specific, structured, clinically precise]

💡 Effective persona formula:

You are a [ROLE] with [EXPERIENCE].

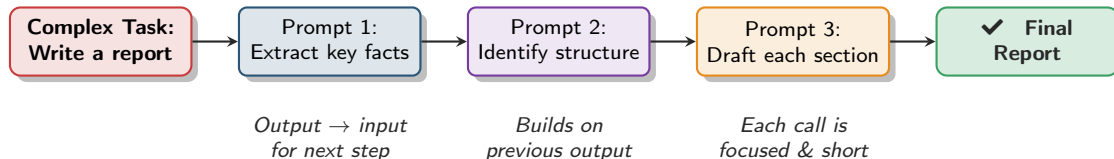
Your audience is [AUDIENCE].

Your communication style is [STYLE].

The model adapts vocabulary, depth, and framing to the persona.

Prompt Chaining & Task Decomposition

✗ One giant prompt trying to do everything → confused, incomplete output.



✓ Each prompt is **focused and verifiable**. The output of one step becomes the input (context) of the next. This is the core idea behind **AI agents**.

Structured Output: JSON Mode & Delimiters

JSON mode

Prompt:
Extract the name and age from:
“Ana Gómez is 28 years old.”
Return as valid JSON.

Output:
{
 “name”: “Ana Gómez”,
 “age”: 28
}

✓ Ideal for downstream code that parses the response. Most APIs support a native `response_format={type: "json_object"}` flag.

XML Delimiters

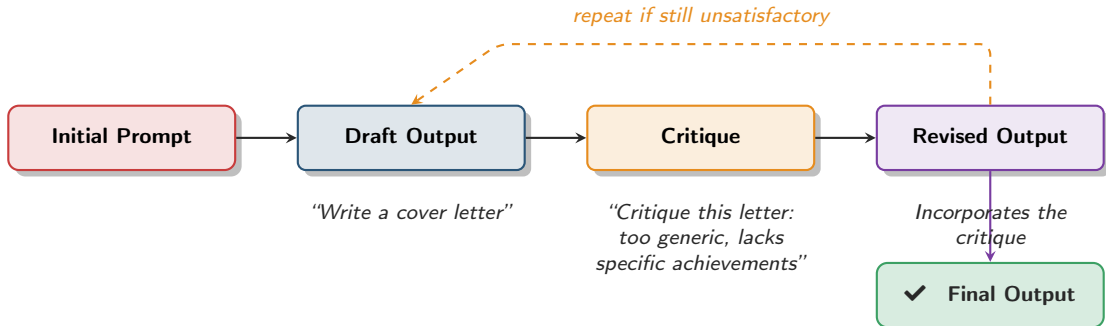
Prompt:
Summarize the article in `<article>` tags.
Return only the summary.

`<article>`
...Long article text here...
`</article>`

Output:
The article discusses...[3 sentences]

💡 Delimiters prevent the model from confusing your data with your instructions. Critical for RAG pipelines.

Self-Critique & Iterative Refinement



Self-Critique Example

Step 1 — Draft

Write a cover letter for a Data Science internship at a fintech startup.
I have 2 years of Python and ML experience.

Step 2 — Critique

Critique the letter above. Identify:

- (a) what is generic or weak,
- (b) missing specific achievements,
- (c) how to make it more compelling.

Step 3 — Revise

Rewrite applying all critiques above.
Keep it under 250 words.

✗ Before

"I have strong Python skills and I am passionate about machine learning. I am a fast learner and a great team player..." *Generic, no evidence, forgettable.*

✓ After

"I reduced fraud-detection inference time by 40% using ONNX quantization, cutting cloud costs by \$2K/month. I want to apply the same approach to [company]'s real-time risk pipeline..."

Specific, quantified, targeted.

You can collapse all 3 steps into **one prompt**:
"Draft, critique your draft, then revise it."

Meta-Prompting: Ask the Model to Design the Prompt

The Idea

Instead of writing the optimal prompt yourself, **ask the model to generate or improve it.**

Common meta-tasks:

- “Write an optimized prompt for task X”
- “Critique this prompt and rewrite it”
- “Add constraints to reduce hallucinations”
- “Adapt this prompt for a non-expert audience”

Example

Meta-prompt:

“I need to classify customer support tickets by urgency (low / medium / high).
Write me an optimized zero-shot prompt for an LLM that handles edge cases and returns valid JSON.”

- ➔ The model generates the ideal prompt.
- ➔ You use *that* prompt in production.

 **The model was trained on millions of prompts** — it knows what makes a good one. Use its knowledge to bootstrap and accelerate your prompt design loop.

Image Prompting: The Art of Visual Descriptions

Anatomy of an image prompt

Subject	What / who is in the image
Style	Photorealistic, oil painting, watercolor, anime, 3D render
Lighting	Golden hour, studio lighting, dramatic shadows
Composition	Close-up, wide angle, bird's-eye view
Quality	4K, DSLR, sharp focus, high detail

Example:

"A Colombian scientist in a modern lab, photorealistic, warm studio lighting, close-up portrait, Canon 5D quality, sharp focus."

— Negative Prompting

Tell the model what to **exclude**:

`negative_prompt: "blurry, watermark, text, cartoon, distorted hands, ugly, low quality"`

💡 Negative prompts prevent the most common artifacts: blurriness, bad hands, watermarks, unrealistic colors.

Key parameters

- **Steps:** 20–50 (more = sharper, slower)
- **CFG Scale:** prompt adherence (7–12 typical)
- **Aspect ratio:** 1:1, 16:9, 4:3
- **Seed:** fix for reproducibility

Prompt Engineering: Cheat Sheet

Technique	When to use	Key trigger phrase
Zero-shot	Simple, well-defined tasks	(just the instruction)
Few-shot	Classification, formatting, style	Input: X → Output: Y
Chain-of-Thought	Math, logic, multi-step reasoning	Think step by step.
Role / Persona	Domain expertise, tone control	You are an expert in...
Structured Output	Downstream parsing, APIs	Return as JSON. Use <tags>.
Self-Critique	Quality-sensitive outputs	Critique your answer, then revise.

Activity & Wrap-Up

Key Takeaways

What we covered

-  GenAI spans text, image, audio, and video — all from a single paradigm shift: **foundation models**
-  LLMs are next-token predictors trained at massive scale
-  Prompting is a **learnable engineering skill**, not trial and error
-  CoT, few-shot, personas, and structured output are your core toolkit

Limitations to remember

- ✗ LLMs **hallucinate** — they generate plausible, not necessarily true content
- ✗ They have a **knowledge cutoff** — no real-time information
- ✗ They are **sensitive to phrasing** — small changes in wording change outputs
- ✗ They can **amplify biases** present in training data

 GenAI is a **tool**, not a replacement for critical thinking. The best results come from humans and AI working together.